

Design-Issue Audit of the NBT Round-Based Revision Pipeline

Cross-agent objective mismatches, non-convergence mechanisms, and scoring-contract defects in a multi-agent manuscript revision pipeline — fourteen findings with file:line evidence, verified probes, and a remediation roadmap.

Z.ai Technical Audit

Repository: backup-092203-7d19d80 · branch main · commit 7d19d80

Scope: nbt_pipeline.py (17,819 lines), nbt_docx_format.py, nbt-skills/, .nbt_test/

Table of Contents

1. Executive Summary	1
1.1 Findings at a glance	1
1.2 Relationship to the repository’s own triage ledger	2
2. The System Under Audit	3
2.1 The scoring contract (judge contract v3)	3
2.2 The ranking key and the eligibility gates	4
2.3 Default session economics	4
3. Cross-Agent Objective Mismatches (A1–A6)	5
3.1 A1 — The em-dash chain: one rule, three bars, no damping	5
3.2 A2 — The writing rubric contradicts the arithmetic that enforces it	6
3.3 A3 — The same defect, four legal classifications	8
3.4 A4 — Verified self-reports versus unverified self-reports, ranked together	8
3.5 A5 — The judge panel’s findings are write-only	9
3.6 A6 — The auditor’s asymmetric agenda and the flag-gated regime change	10
4. Convergence and Cost Dynamics (B1–B5)	10
4.1 B1 — No cross-round scale, and the incumbent is re-rolled every round	10
4.2 B2 — The anti-regression gate guards the wrong edge	11
4.3 B3 — The field accumulates ancestors	11
4.4 B4 — The rewrite stage’s feedback deliverable is dead	12
4.5 B5 — Integration: conservative by rule, redundant in execution	12
5. Code-Level Defects and Test-Suite Results (C1–C3)	13
5.1 C1 — scan_format_in_sources drops the operator policy (verified)	13
5.2 C2 — Tier vocabulary drift between the skill and the scanner	14

5.3 C3 — Test-suite execution results 14

6. Remediation Roadmap **15**

6.1 Sequencing and priorities 17

6.2 What not to change 17

7. Verification Log **17**

1. Executive Summary

This report audits the design of the NBT round-based revision pipeline as shipped in repository backup-092203-7d19d80 (branch main, commit 7d19d80). The pipeline drives a multi-agent loop - blind rewrites, a frozen review, an optional auditor, finding-driven revisions, whole-pool integrations, and a blind pairwise judge panel - that is intended to improve a Nature Biotechnology manuscript package round over round. The audit was commissioned to look specifically at the classes of failure the operator hypothesised: prompts to the codex agents that encode different scoring philosophies than the functions that actually consume the agents' work, non-convergence or slow convergence of scores as rounds accumulate, mismatches between what the judges perceive as better or worse and what the other agents act on, and structural blind spots of the judge panel for semantic, syntactic and grammatical defects.

The method was a full static read of the orchestrator (nbt_pipeline.py, 17,819 lines), the OOXML scanner (nbt_docx_format.py, 3,047 lines), both bundled skills (nbt-review, nbt-revise) and their prompt templates, cross-checked against the repository's own triage ledgers (NBT_DESIGN_TRIAGE_LEDGER.md and companions, roughly 245 KB) so that known and deliberately accepted behaviours are marked as overlaps rather than presented as new. Every behavioural claim below carries a file:line citation into the audited tree. Where a claim could be executed, it was: a runtime probe demonstrates the policy-shadowing defect (finding C1), and all 31 offline regression suites in .nbt_test/ were run in this environment (finding C3 reports the two environment-dependent failures). No claim in this report rests on inference alone where code or a probe could decide it.

Fourteen findings are reported. Three are headline mechanisms that jointly explain the operator's symptoms. First, the pipeline mandates large amounts of revise-stage work in defect classes the judge contract prices at exactly plus or minus one point and forbids from ever being decisive (A1, A2) - so the properties the pipeline works hardest on are the ones the scoreboard cannot see. Second, the highest-quality information the system produces - the blind panel's per-comparison defect ledgers - is write-only: it collapses into one integer per pair and is then discarded, while the revision loop is steered only by the non-blind review channel (A5, B4). Third, the round-over-round selection statistics are deliberately non-comparable across rounds, the incumbent is re-judged from scratch by fresh noisy sessions every round, and the anti-regression gate guards the pristine original rather than the incumbent champion - together these permit noise-driven churn and true round-over-round regression (B1, B2, B3).

14 findings, each with file:line evidence	76% of default-run sessions are judge sessions	+/- 1 the judge cap on the classes the prompts mandate work in
---	--	--

1.1 Findings at a glance

ID	Finding	Severity	Primary surface
A1	The em-dash chain: revise-side mandate, rewrite-side freedom, judge-side cap of +/-1, and a dead operator knob	High	M20 rules, scan tiers, judge caps
A2	Writing rubric says 'ten unfixed rows are ten rows'; the arithmetic caps the class at one point	High	WRITING_RUBRIC vs TIER_CAPS
A3	The same defect is classifiable into four different tiers by judgement call; nothing measures inter-judge class agreement	Medium	DEFECT_CLASS_RULE, panel diagnostics
A4	Champion tie-breaks rank a verified self-report against unverified self-reports from competing arms	High	candidate_tiebreak_inputs, ranking key
A5	The judge panel's findings are write-only: no channel from judge ledgers into any revision stage	High	postcheck_judge, aggregate_round
A6	Auditor agenda is asymmetric (undecidable must confirm; additions only) and the reviser's re-verification regime flips with a setup flag	Medium	AUDIT_DIRECTIVES, E7, R1
B1	No cross-round score comparability; the incumbent is re-judged by fresh sessions each round, so champion identity near the top is a random walk	High	aggregate_round, score_model_doc
B2	The anti-regression gate guards the pristine original, not the previous champion; vs_base is explicitly not a ranking input	High	anti_regression_verdict, select_champion
B3	The judge field accumulates every prior champion: signal dilution plus quadratic judge-cost growth per round	High	build_field, judge workload
B4	The rewrite stage's declared feedback deliverable (PROBLEMS SURFACED) is stripped from every downstream corpus and view	High	BOOKKEEPING_FILES, build_corpus_dir
B5	Integration is conservative by rule (shared defects untouched), executed K times redundantly, and scored with unverifiable tie-break inputs	Medium	INTEGRATE_DIRECTIVES rule (d)
C1	scan_format_in_sources overwrites its policy parameter with module defaults - the operator's format policy never reaches any seeded scan or gate	High	nbt_pipeline.py:6020-6021
C2	Tier vocabulary drift: sweeps.md calls em-dash rows 'findings'; the code-side tier column says 'advisory'	Medium	sweeps.md:637 vs tier_of()
C3	Test-suite hermeticity: one check hardcodes file mode 0o644; two checks assume WSL (wslpath/cygpath) without a skip guard	Low	test_judge_blinding.py:298; C16

Table 1: The fourteen findings. Severity reflects impact on the operator's stated symptoms: score convergence, cross-agent consistency, and cost.

1.2 Relationship to the repository's own triage ledger

Two findings partially overlap the repository's own triage ledger and are reported here with that overlap marked. Finding A4 extends ledger item C22 (NBT_DESIGN_TRIAGE_LEDGER.md:234-241), which accepted self-reported tie-breaks 'by design' for exact ties; the new element is the structural asymmetry that only the revise arm's report is cross-checkable while it competes in the same sort against arms whose reports cannot be checked at all. Finding B1's non-comparability is self-documented in score_model_doc (nbt_pipeline.py:15207-15210); the new elements are the re-judging noise mechanism and its interaction with the exact-tie incumbent rule. Everything else in this report is, to the best of a full-text search of the three ledgers, not previously recorded.

2. The System Under Audit

Each round r materialises a base ($a1_r$: the pristine original in round 1, the previous round's pinned champion afterwards), M full rewrites ($w1..wM$) staged before any review exists, one frozen review of the base, N revisions ($a2..a\{1+N\}$) that consume the frozen or audited findings, and $K = 1+M+N$ integrations - one per pool member, each merging the whole pool into its own base. The round's field is the deduplicated union of the pristine original, every prior pinned champion, and all fresh arms; a blind judge panel of J sessions per field member scores each member against every other on an integer $-4..+4$ scale. The champion is pinned, published, and becomes the next round's base. Default configuration: 2 rounds, judges [3], rewrites [2,1], revises [1,1], integrators all (nbt_pipeline.py:757-772).

Two structural properties matter for everything that follows. First, the review sweeps the base only (REVIEW_DIRECTIVES, nbt_pipeline.py:2730-2731) - defects that exist only in a rewrite or integration candidate are never reviewed, only judged. Second, the judge sandbox is blinded absolutely: no review findings, no orchestrator scans, no provenance vocabulary (README.md, 'The judge gets none of it - blinding is absolute'), which is sound practice and precisely why the judge channel would be the most valuable source of unbiased defect signal - if anything consumed it (finding A5).

2.1 The scoring contract (judge contract v3)

Judge contract v3 (JUDGE_CONTRACT_VERSION, nbt_pipeline.py:1398) derives every comparison integer from a per-comparison ledger of resolved and introduced items. Row weights are minor 1 / major 2 / critical 3, but each tier's net contribution is capped, and the two lowest tiers are minor-only by policy: a formatting or writing row with severity 'major' is a schema error that fails the judge's run (judge_basis_problems, nbt_pipeline.py:12544-12547). The orchestrator re-derives the integer from the ledger and discards any sheet whose number contradicts it (derived_comparison_score, nbt_pipeline.py:12402-12432). This is a well-engineered calibration straightjacket - and it is also the mechanism behind findings A1 and A2: the straightjacket prices the pipeline's own mandated work classes out of decidability.

Tier (priority order)	Row weight (min/maj/crit)	Net cap per tier	Minor-only?	Where the revise stage's mandatory work lands
correctness	1 / 2 / 3	+/-4	no	-
consistency	1 / 2 / 3	+/-3	no	-
preservation	1 / 2 / 3	+/-3	no	-
completeness	1 / 2 / 3	+/-2	no	-
formatting	1 / 1 / 1	+/-1	YES	M20 editorial rows (em-dash density, quote style, journal italics), M18 caption compression
writing	1 / 1 / 1	+/-1	YES	L1-L11 language pass (grammar, punctuation, register, stiff phrasing, segmentation)

Table 2: Tier weights and caps (nbt_pipeline.py:1404-1413) against the revise stage's mandatory work classes.

2.2 The ranking key and the eligibility gates

The ranking key is (-median, -mean, IQR, critical_remaining, writing_remaining, digest, id) over each member's flat list of $2 * J * (|field| - 1)$ directed scores (select_champion, nbt_pipeline.py:13307-13309). Eligibility requires a complete panel for every row and, for fresh arms only, an anti-regression verdict against the pristine original: `vs_original median >= 0` under the default rule (`VS_ORIGINAL_TOLERANCE = 0.0`, nbt_pipeline.py:2453; `anti_regression_verdict`, :2483-2499). An exact tie on (median, mean, IQR) retains the incumbent base (:13312-13324). The head-to-head statistic against the round's own base (`vs_base`) is computed, reported with win/loss counts and a sign-test p-value - and then explicitly excluded from the ranking (:13297-13304, and `score_model_doc` :15219-15231). The two counts that DO break statistical ties, `critical_remaining` and `writing_remaining`, are self-reported by the very session that produced the candidate (finding A4).

2.3 Default session economics

Under default settings the two-round pipeline materialises 59 agent sessions: 8 producing sessions in round 1 (2 rewrites, 1 review, 1 revise, 4 integrations) and 6 in round 2, plus 45 judge sessions (24 in round 1 for an 8-member field, 21 in round 2 for a 7-member field). The judge wave is 76 percent of all sessions, and its unit of work is the most expensive in the system: each judge session must run the frozen sweep set (M1-M17, M18-M24, J1-J4) on its own target, read every opponent in full (`JUDGE_DIRECTIVES` :3769-3783), dispose 25 frozen check ids per opponent (`JUDGE_COVERAGE_CHECKS` :10433-10439 - 4,200 dispositions in round 1 alone), and render pages for the visual pass. Because the field grows by one unique-content member per round that crowns a new champion (`build_field` :9798-9832), judge cost grows quadratically in field size and cubically across a run - charted below (finding B3).

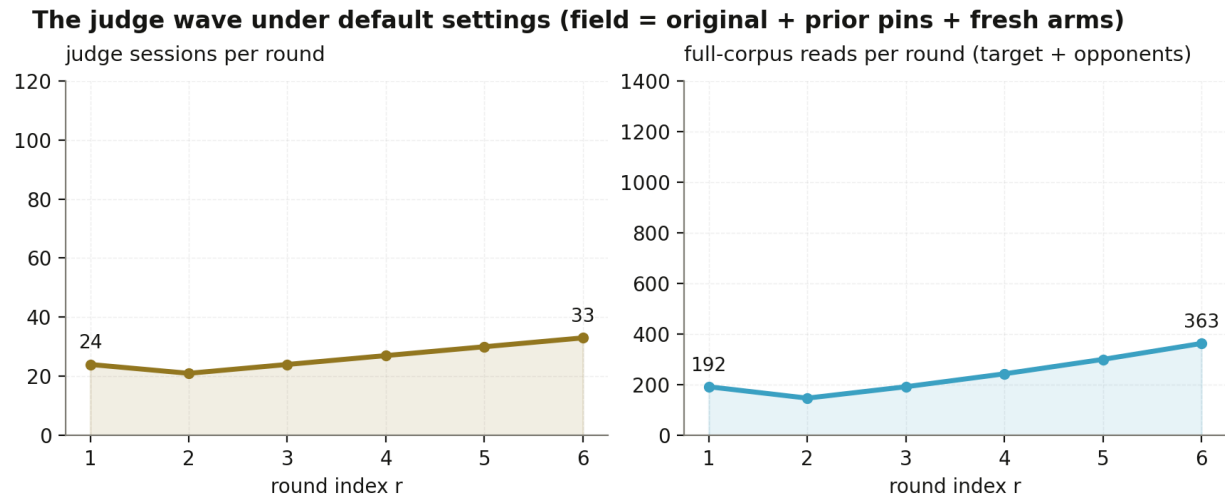


Figure 1: Judge-wave growth under default settings. The field gains one unique-content member per round that crowns a new champion (build_field, nbt_pipeline.py:9798-9832); each field member is judged by 3 sessions that each read the whole field.

3. Cross-Agent Objective Mismatches (A1–A6)

The six findings in this group are the audit's core answer to 'different prompts to codex match different scoring functions'. They are presented in causal order: what each agent is told to optimise, what the scoring code can actually reward, and what happens where the two disagree.

3.1 A1 — The em-dash chain: one rule, three bars, no damping

The chain, end to end. The code-side scanner prices em-dash density above 2.0 per 1,000 words as a medium-severity row with fix kind 'editorial' (nbt_docx_format.py:2152-2160, policy default :74). The review skill the agents read instructs that this row is 'a finding (editorial: the revision arm rewrites; never a mechanical replacement of meaning-bearing text)' (nbt-skills/nbt-review/references/sweeps.md:637-639), and the orchestrator's seeded M20 artifact repeats that 'the editorial rows (em-dash density, quotation style, journal-italic policy) are findings for the revision/integration arms' (nbt_pipeline.py:6373-6374). The revise prompt then makes the fix mandatory: 'Fix the EDITORIAL rows yourself: rewrite parenthetical em-dashes as commas/parentheses' (M20_REVERSE_RULE, nbt_pipeline.py:1854-1856). So the revise arm is contractually obliged to spend its edits on em-dash reduction.

The reward side, however, is priced at almost nothing. FMT-P1 (em-dash density) is not in FINDING_TIER_RULES (nbt_docx_format.py:595-608), so tier_of() classifies it 'advisory' (:611-613) - the seeded table's own tier column says advisory while the same artifact's header text calls the row a finding, a vocabulary split the reviewer and the auditor must resolve by taste (finding C2). The judge sweep allows a formatting preference of 'at most +/-1' and 'never decisive' (M20_JUDGE_SWEEP, nbt_pipeline.py:1872-1883); the judge contract enforces minor-only rows and a +/-1 cap for the whole formatting tier (TIER_CAPS :1410-1411, MINOR_ONLY_TIERS :1413). And the anti-regression gate that guards every package-producing stage counts finding-tier rule families only (scan_regression_problems :11617-11650) - an advisory row is invisible to it. Net effect: a rewrite arm that

INCREASES em-dash density passes every gate, and the revise arm that dutifully removes them can earn at most one point for it.

Fix the EDITORIAL rows yourself: rewrite parenthetical em-dashes as commas/parentheses, make the quotation style uniform, keep journal titles italic only in the reference list... (M20_REWRITE_RULE, nbt_pipeline.py:1853-1856)

The rewrite side has no counter-pressure at all. M20_REWRITE_RULE tells the rewriter formatting is 'REPORTED here, not fixed... keep the base's formatting conventions' (nbt_pipeline.py:1866-1871) - but em-dash density is not a 'convention' in the sense the rule's examples use (legend spacing, quotation style, URL treatment), and nothing in the rewrite postcheck or the regression gate notices an increase. Since rewrites are full-document rewrites by an LLM whose prose prior freely produces em-dashes, the round-over-round dynamics are: rewrites add em-dashes, revisers remove them, judges can express at most one point of preference either way, and the tie-break counts (writing_remaining) count category-2 findings only - an M20 row is category 4. The operator's hypothesised conflict (judges penalise em-dashes, revisers favour them) is real in this weakened form: the mandate is one-sided, the reward is capped at noise level, and no mechanism damps the oscillation.

The operator's knob is dead in the seeding path. POLICY_DEFAULTS exposes max_em_dashes_per_1000 for the operator (setup --format-policy). The scanner honours it (probe below); the mechanical fixer honours it (fix_docx_in_place -> fix_package, nbt_pipeline.py:6104-6138). But every SEEDED scan - the review's M20 table (_seed_review_format_artifact :6352-6356), the evidence packs (:6437), the no-new-defect-family gate (:11636) and the decide-stage winner re-scan (:16195) - silently uses module defaults, because scan_format_in_sources overwrites its policy parameter before using it (finding C1). And the judge, blinded by design, scans with CLI defaults. One rule, three different bars in a single run.

Probe (scripts/probe_policy_shadow.py, this audit): a synthetic .docx with 6 em-dashes in 23 words (density 261/1,000) - analyse_package() with default policy reports FMT-P1; with max_em_dashes_per_1000 = 500 it reports no FMT-P1 (the scanner honours the policy); the wrapper scan_format_in_sources(sources, policy=...) however discards the parameter at nbt_pipeline.py:6020-6021 ('policy = dict(getattr(mod, POLICY_DEFAULTS...)); policy.update(policy or {})' - the second line updates the dict with itself.

Probe output (this audit, scripts/probe_policy_shadow.py): the scanner honours the operator cap; the wrapper drops it. See finding C1.

3.2 A2 — The writing rubric contradicts the arithmetic that enforces it

The prompt and the arithmetic contradict each other in the same contract. The writing rubric that every judge receives says: 'The tier is minor-only, so it can separate two otherwise equal packages by at most one point -- but every row counts, and ten unfixed rows are ten rows' (WRITING_RUBRIC, nbt_pipeline.py:11567-11568). The enforcement code says the opposite in effect: ten minor writing rows

produce a net of 10, the writing tier cap clamps it to 1 (TIER_CAPS :1410-1411), and derived_comparison_score (:12402-12432) fails any sheet whose integer disagrees with the clamped result. A judge who believes the rubric's plain words and writes +3 for a package that fixed ten grammar defects has its sheet discarded and its session retried - the marginal judge-visible value of the second through tenth grammar fix is exactly zero.

The tier is minor-only, so it can separate two otherwise equal packages by at most one point -- but every row counts, and ten unfixed rows are ten rows. (WRITING_RUBRIC, nbt_pipeline.py:11567-11568; enforced cap: TIER_CAPS writing = 1, :1410-1411)

The revise stage is nonetheless contractually obliged to do that work. Every package-producing stage must run the L1-L11 language pass sentence by sentence (LANGUAGE_PASS_RULE, nbt_pipeline.py:3658-3684), where L10 is grammar and L11 is typos/punctuation, with a re-scan after every step; the writing tier of the judge rubric maps one-to-one onto those same checks (Q10 grammar, Q11 typography, :11561-11563). So the pipeline's own mandates concentrate effort precisely in the classes the scoreboard prices at one point and forbids from deciding anything. This is the exact sense in which 'the judges ignore semantic/syntax/grammar errors': not by inattention, but by contract - minor-only rows, a +/-1 cap, and 'never decisive' (JUDGE_DIRECTIVES :3873-3878). A package with fifty open grammar defects and one with none are at most one point apart, and that point is cancelled by any single minor correctness item anywhere else.

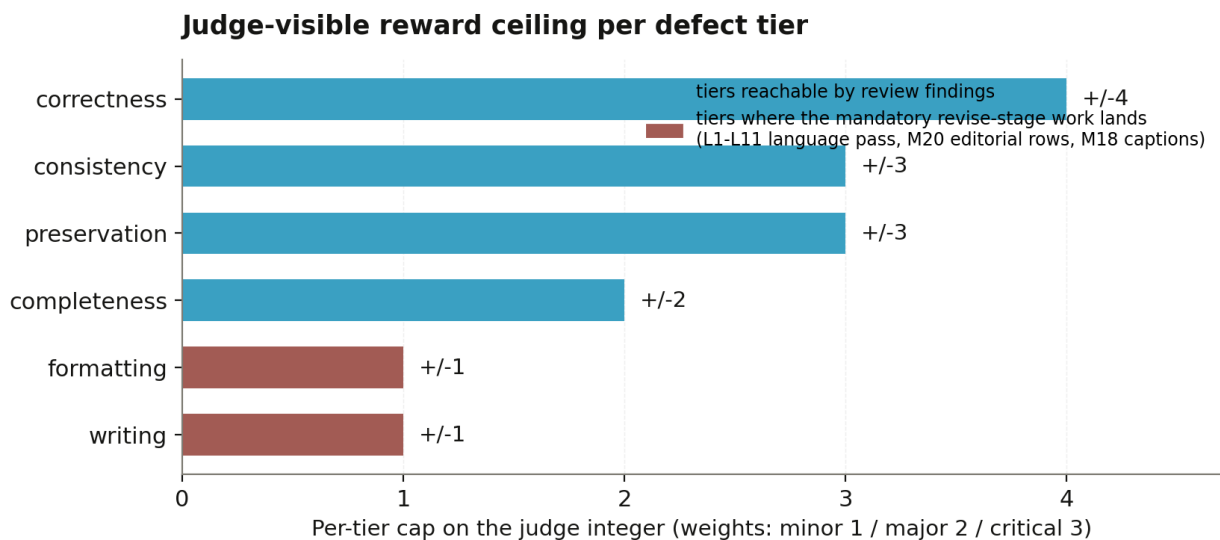


Figure 2: Where the mandated work lands versus what the scoreboard can express. The revise stage's mandatory passes (L1-L11, M20 editorial rows, M18 captions) sit in the two tiers the judge contract caps at one point and forbids from deciding comparisons.

The severity funnel makes genuinely major language defects unrepresentable. A grammar problem that changes meaning must be classified correctness to be scored as major (category-2 mapping, DEFECT_CLASS_RULE :2072-2078) - a judgement call finding A3 shows is uncontrolled. A grammar problem that does not change meaning must be written as a MINOR writing row or the sheet fails (judge_basis_problems :12544-12547). There is no legal way for a judge to say 'this package has a

grammatical catastrophe': the vocabulary has no rung for it. The one-sentence scale anchor '+1 slightly better - a small but REAL net advantage (never a cosmetic one)' (:3835) sits oddly next to a cap that makes the difference between one and five hundred fixed typos invisible.

3.3 A3 — The same defect, four legal classifications

The shared defect vocabulary assigns category-2 (writing quality) defects to one of four classes 'by the concrete defect' (:2072-2078): consistency when something is said two ways, correctness when the wording changes the meaning, formatting for one-off preferences, writing for grammar that changes no meaning. The four classes carry caps of 3, 4, 1 and 1 respectively - so the classification decision alone can swing a comparison's derived integer by three points. Two judges reading the same ambiguous sentence (a dangling modifier that subtly shifts a claim; a 'term spelled two ways' that one judge reads as a convention and another as a typo) produce materially different integers, and both sheets pass validation, because the validation checks each sheet's arithmetic against its OWN ledger, never judge-to-judge agreement.

The panel-quality diagnostics confirm the gap: `aggregate_round` computes `zero_share`, mean absolute score, within-direction range, direction flips between opposite pairs, per-judge means, `basis_counts` and ledger sign conflicts (`nbt_pipeline.py`:13029-13092) - all reported, none used for eligibility, and none of them measures whether two judges classified the same evidence into the same tier. 'Mismatch between what the judges perceive as better/worse and what other agents perceive' is therefore not only unmeasured between the panel and the other agents; it is unmeasured within the panel itself, and the derived-score straightjacket converts every classification disagreement directly into score noise rather than flagging it.

3.4 A4 — Verified self-reports versus unverified self-reports, ranked together

When the panel statistics tie, the champion is decided by two counts the producing agent reports about its own package: `critical_remaining` and `writing_remaining`, ranked AFTER (median, mean, IQR) and BEFORE the content digest (`select_champion` :13307-13309). The design is honest about this - `score_model_doc` says the counts are 'SELF-REPORTED by the producing agent (cross-checked against the frozen review only for the revise arm and only in the reported-zero direction)' (:15222-15226), and the integrate prompt itself admits 'This run has no frozen review to cross-check against - only the revise arm consumes one' (:3383-3385). But the honesty does not fix the incentive geometry: the only arm whose under-reporting is detectable (revise, warned when it claims zero while the frozen review lists findings) competes in the same sort against rewrite and integration arms whose equivalent claims cannot be checked at all. On an exact statistical tie, an honest revise arm reporting `writing_remaining` = 2 loses to an integration arm self-reporting 0 with the same real defect count.

The cross-check that does exist is one-directional and one-armed by construction: `critical_findings_input` and `writing_findings_input` read the frozen review from the run's own sandbox (`nbt_pipeline.py`:10314-10349), a directory only the revise arm possesses. The ledger's own triage accepted this as design (C22, `NBT_DESIGN_TRIAGE_LEDGER.md`:234-241: 'self-reported data still breaks exact ties by design'). The finding here sharpens rather than repeats it: within one ranking, verified and unverifiable self-reports are compared as if equivalent, the sentinel for missing values (+inf, `_valid_count` :13164-13175) punishes absence while rewarding confident fabrication, and the counts also interact with

the auditor (finding A6): an auditor severity raise moves a defect into critical_remaining without any judge ever seeing it.

3.5 A5 — The judge panel's findings are write-only

Every judge session produces three artefacts: scores.json (one integer, one basis tier, two ledger lists, and a 25-id check-disposition map per opponent), the judge_review/ sweep artifacts (the frozen-sweep review of the blinded target, including its own M18/M19/M20 tables), and the visual-inspection record. The orchestrator consumes exactly one of these: the integers. aggregate_round reads the comparisons for scores (:12900-12917) and nothing else; postcheck_judge validates the sheet and writes the orchestrator's own re-scan of the blinded target to reports/judge_evidence_.json for the human (:12683-12709) - an auditability channel, not an agent channel. The introduced lists - the concrete defects the blind panel found in the version that WON - have no consumer: no prompt names them, no materializer copies them, and the next round's review is a fresh nbt-review pass over the champion that reconciles only the previous REVIEW's findings (PRIOR_ROUND_RULE :10738-10755).

This is the deepest form of the operator's 'mismatch between what the judges perceive and what other agents act on'. The revision loop's only steering inputs are the review's findings (non-blind, base-only) and the auditor's dispositions of those same findings. A defect the panel sees consistently but the review skill misses is re-discovered or missed from scratch every round, at the price of a full judge session each time, and the aggregate signal that would prioritise it (how often did the champion's judges list it as introduced?) is computed nowhere. The information cost is quadratically expensive to collect and free to throw away: by round r , the panel has produced $J \cdot (|field| - 1)$ ledgers about the incumbent's defects alone, and the pipeline's memory of them is a single median.

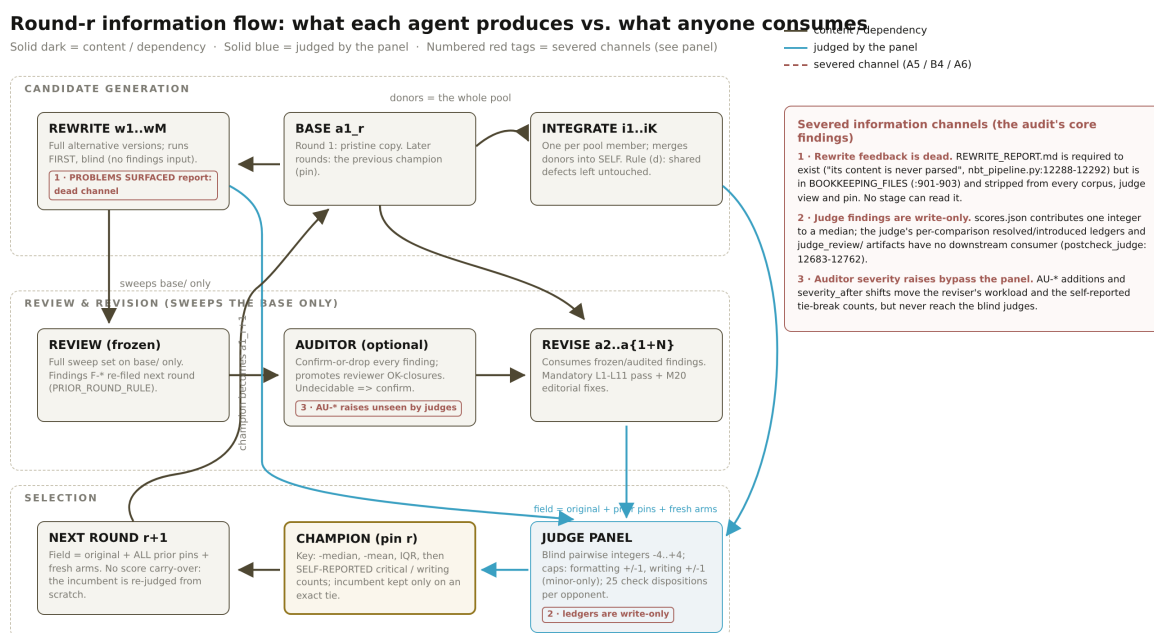


Figure 3: Round-r information flow. Solid dark arrows are live channels; blue marks what the panel judges; the numbered red tags mark the severed channels established by findings A5 (2), B4 (1) and A6 (3).

3.6 A6 — The auditor's asymmetric agenda and the flag-gated regime change

The auditor's brief is asymmetric by design, and the asymmetry points the same way as the failure the operator described. Every frozen finding must be confirmed or dropped with evidence, and 'A finding you cannot decide from the corpus is confirm with a note - dropping on cannot tell is forbidden' (AUDIT_DIRECTIVES, nbt_pipeline.py:3135-3137; enforced in audit_artifact_problems :11712-11760, where a drop needs a 20-character reason and 40-character evidence). Task 2 then directs the auditor to 'ATTACK THE DISPOSITIONS': every finding-tier row the reviewer closed as OK is re-checked and 'a real defect becomes a new AU-* finding' (:3138-3144). apply_audit_to_findings composes the revisers' list as kept + adds (:11795-11807). So across the review-audit boundary the list can only grow (minus evidenced drops), and across rounds PRIOR_ROUND_RULE re-files still-present findings with fresh ids - a ratchet in which an item the reviewer judged fixed-but-marginal can return as an auditor add with a raised severity (the schema carries severity_after, :3155), and 'a fix in the eye of the reviewer' genuinely can become 'a bigger defect in the eye of the auditor'.

The reviser's own regime flips on a setup flag. With audit off - the default (README.md:76-77; audit_enabled :1520-1527) - the revise prompt's R1 step re-verifies every finding and lets the reviser discard false positives with a recorded rationale (REVISE_DIRECTIVES :2921-2924): the reviser is its own auditor. With audit on, edit rule E7 forbids re-litigating drops ('a DROPPED finding is not yours to re-litigate', nbt-skills/nbt-revise/references/edit_rules.md:243-259). The same frozen list is therefore filtered by different agents under different bars in different configurations, and the auditor's adds and severity raises shift the self-reported tie-break counts (A4) without ever reaching the blind panel - the judges cannot see that a defect was re-classified critical after the fact.

4. Convergence and Cost Dynamics (B1–B5)

The operator asked why scores do not converge as rounds increase and why convergence is slower than the session budget should buy. Five mechanisms jointly answer both questions; none of them is a bug in the ordinary sense - each is a documented, deliberate design choice whose interaction removes the properties the operator expects.

4.1 B1 — No cross-round scale, and the incumbent is re-rolled every round

The score model itself declares the absence of a cross-round scale: 'per-round champion statistics come from different panels and different fields and are NOT comparable across rounds; the vs_base margin of the final round is the only paired statement the decision supports' (score_model_doc, nbt_pipeline.py:15207-15210). There is therefore no trajectory to converge: each round's median measures 'record against this round's field', and the field changes every round (B3). What the operator observes as non-convergence is the design working as documented - which is precisely why the documentation deserves to be counted a design issue rather than a defence.

Worse, the incumbent's qualification is re-rolled from scratch every round. `aggregate_round` reads only the current round's judge runs (`ctx.runs(kind='judge', round_no=r)`, :12866), so the pin that won round `r` is re-judged by `J` fresh sessions in round `r+1` against a partly-new field; its median, mean, IQR, panel completeness and anti-regression verdict are all new random variables even though its content is unchanged. The pipeline has content fingerprints for exactly this purpose (`corpus_content_fingerprint` :8442-8488, used for field dedup) but no verdict cache: a pair of content-identical versions (the original and an unchanged pin) is re-judged $2 \times J$ times per round, every round, at full session cost, adding noise rather than information. The incumbent-retention rule only guards EXACT ties on (median, mean, IQR) (:13312-13324): a base that won round `r` with median +1 can sample median 0 in round `r+1` by judge noise alone and be replaced by a challenger that is not measurably better. Near the top of the field, where candidates are genuinely near-identical, champion identity performs a random walk. The production evidence in the tree supports the concern: the 2026-09-17 panel's 294 directed scores used only -3..+3 and never the +/-4 runs (:1390-1393) - the scale's headroom is unused, so ties are common.

4.2 B2 — The anti-regression gate guards the wrong edge

The only anti-regression eligibility gate compares a fresh arm against the PRISTINE ORIGINAL, never against the incumbent champion: `anti_regression_verdict` fails a candidate when its `vs_original` median is negative (`VS_ORIGINAL_TOLERANCE` = 0.0, :2453, :2483-2499). The head-to-head against the round's own base is computed (`vs_base` :12958-12963), reported with sign counts and an exact one-sided sign-test `p` (:12988-12992), and then explicitly excluded: 'vs_base is reported but NOT a ranking key: its $2 \times$ judges directed scores are few enough that one outlier session flips the sign statistic' (:13297-13304). The rationale (a noisy $2J$ pair should not decide) is defensible in isolation; the composition is not. A challenger that the panel scores BELOW the incumbent base (negative `vs_base` median) but above the original can still win the field-wide median ranking, because the field-wide median and the head-to-head are different statistics over different opponent sets.

Concretely: the base tends to score well against the original (it is a descendant with many fixes) but only middling against diverse fresh mutants; a fresh mutant can beat the weak and stale field members by wider margins than the base does, winning the round while losing the pair. The round then crowns a version the panel ranked below the version the round started from - a real round-over-round regression that no gate catches, and a direct mechanism for the 'scores do not converge' symptom. The operator's alternative (require non-negative `vs_base`, or rank on `vs_base` first with the sign test as the guard) was considered and rejected in the code comments for the noise reason; the audit's position is that a noisy gate is still better than no gate when the alternative is silent regression, and that the fix section proposes a middle path (a composite eligibility rule) that keeps the noise argument intact.

4.3 B3 — The field accumulates ancestors

`build_field` composes the round's field as {original} UNION {ALL pins from rounds < `r`} UNION fresh arms, deduplicated by content fingerprint with first occurrence winning (:9798-9832). Every round that crowns a new champion permanently adds one member to every future field. Three consequences follow. Cost: the judge wave runs `J` sessions per field member, each reading the whole field and disposing 25 check

ids per opponent - $O(|\text{field}|^2)$ full-corpus reads per round, growing to $O(R^3)$ across a run; under defaults the two-round pipeline already spends 76 percent of its 59 sessions on judging, and round 6 of the same configuration would judge a 10-member field (30 sessions, 300 corpus reads in that round alone) - the chart below plots it.

Signal: each member's median mixes comparisons against live rivals with comparisons against stale ancestors that every live candidate beats. As the stale share grows, all live candidates' medians converge to the same easy-win value, the median stops discriminating among the candidates that matter, and selection degenerates through the mean and the IQR into the self-reported counts of finding A4 - the ranking's effective keys become the least verifiable data in the system. Re-judging: the unchanged pairs (original vs pin_k) are re-sampled by fresh sessions every round (B1); with $J=3$ this is six fresh directed scores per unchanged pair per round - pure noise re-sampling at the most expensive unit of work the pipeline has. The operator's 'too much scoring' hypothesis is confirmed with a number: at round 6, roughly 42 percent of all judge-session comparisons in that round are against versions that existed unchanged in previous rounds.

4.4 B4 — The rewrite stage's feedback deliverable is dead

The rewrite stage's stated purpose has three clauses: provide diversity, surface new problems, and 'serve as feedback/reference for the rest of the round and the author' (REWRITE_DIRECTIVES, nbt_pipeline.py:3468-3470). The problems list is a required deliverable: REWRITE_REPORT.md section 3, 'PROBLEMS SURFACED: every ambiguity, unclear antecedent, perspective/voice drift, inconsistency, confusing passage or conflicting number you hit while rewriting... This list is the stage's feedback/reference deliverable for the later stages and the author' (:3569-3574). Architecturally, no later stage can read it. rewrite_report.md is in BOOKKEEPING_FILES (:901-903); build_corpus_dir strips bookkeeping from every pin, published winner, integration donor and next-round base (:8614-8631); the judge view excludes it by the same list (_is_view_excluded_file :992-1040). The postcheck requires the file to exist and states that 'its content is never parsed for scores' (:12288-12292).

So M full-rewrite sessions per round - the most expensive producing arms after integration - generate a defect list whose only reader is the human operator. The review cannot use it (it reviews the base, :2730-2731); the revisers cannot use it (they receive the frozen review only); the integrators cannot use it (donors are stripped corpora); the judges cannot use it (blinding, correctly). The problems a rewrite surfaces must be independently re-found by the next round's review or not found at all - and defects present ONLY in a rewrite candidate (not in the base) are seen by no reviewing agent at all, only by the blind judges, whose findings are write-only (A5). Two of the pipeline's information channels are thus severed at the same point in the graph; the diagram in chapter 3 shows both.

4.5 B5 — Integration: conservative by rule, redundant in execution

The integration contract is deliberately conservative. Rule (d): 'a defect present in the base AND in at least one donor -> LEAVE UNTOUCHED (do not improve the shared majority...)' (:3245-3249). Conflicts resolve toward the base (:3259-3260). A port must be 'demonstrably better under the source hierarchy' to happen at all. The consequence is that the one merge the user's hypothesis cares about - fixing a defect every candidate shares - is exactly what the integrator is forbidden to do; shared defects can only leave the

pool through the review-revise channel, which reviews the base only (B4). Meanwhile the execution is redundant: $K = 1+M+N$ integration sessions per round, each reading the whole pool (materialize_integrate :9585-9642), each producing a near-duplicate of the others (all import the same demonstrably-better fixes from the same donors), and the content dedup only removes EXACT matches (:9817-9831) - so K near-identical packages each get a full judge panel row. Under defaults that is 7 of round 1's 8 producing sessions producing candidates the panel can barely tell apart, priced at 4,200 check dispositions to adjudicate.

The strategy the operator called 'merging into a base is suboptimal' is therefore suboptimal in a precise sense: the merge explores K parallel near-identical baselines instead of one canonical merge, forbids itself the majority-defect fixes that would differentiate the result, and reports its tie-break counts through the unverifiable self-report channel (A4). A single canonical integration (one session merging the pool into the round's base) plus one variant arm would hold the diversity argument at one quarter of the integration sessions and remove the near-duplicate judging load.

5. Code-Level Defects and Test-Suite Results (C1–C3)

Two code-level defects and one test-suite observation complete the audit. Both defects were verified by execution in this environment; the probes are reproduced in the appendix so the operator can re-run them against any tree.

5.1 C1 — scan_format_in_sources drops the operator policy (verified)

scan_format_in_sources accepts a policy parameter and then destroys it: 'policy = dict(getattr(mod, POLICY_DEFAULTS, {}) or {})' rebinds the name to the module defaults, and 'policy.update(policy or {})' updates the dict with itself - a no-op (nbt_pipeline.py:6020-6021). Every caller that passes an operator-configured policy is silently ignored: the seeded M20 review artifact (:6352-6356), the code-side evidence packs (:6437), the before/after scans recorded around every FORMAT-FIX (:6152, :6161), the no-new-finding-tier-family regression gate (:11636), the setup-time original scan (:14720) and the decide-stage winner re-scan (:16195). The mechanical fixer path honours the real policy (fix_docx_in_place -> mod.fix_package, :6104-6138), so the pipeline can FIX under one policy while SEEDING and GATING under another.

```
def scan_format_in_sources(sources: list, policy=None) -> dict: # :5999
    ...
    policy = dict(getattr(mod, "POLICY_DEFAULTS", {}) or {}) # :6020 <-- parameter
    overwritten
    policy.update(policy or {}) # :6021 <-- self-update no-op
```

Concrete effects. (1) The em-dash knob of finding A1 is dead in every seeded path: an operator who raises max_em_dashes_per_1000 to 10 still gets finding rows seeded at the default 2.0 and a revise arm instructed to remove them. (2) An operator who sets title_page_header to 'keep' still gets FMT-S3 rows seeded - and FMT-S3 IS finding-tier (nbt_docx_format.py:597), so it also feeds the regression gate that can fail a run.

(3) The `FORMAT_FIX` artifacts are internally inconsistent: the before/after scans describe a default-policy world while the fixer applied operator policy, so a 'fix' can leave rows the scan still reports and the operator cannot tell whether the fixer failed or the scan is describing a different policy. The one-line fix is to build the effective policy before the rebind - see the roadmap; a regression test should assert that a seeded scan honours a custom policy.

5.2 C2 — Tier vocabulary drift between the skill and the scanner

The review skill's M20 section - which the reviewer, the reviser and the judge are all told to read as the frozen checklist - says: 'mixed straight/curly quotation marks, a spaced hyphen used as a dash, or em-dash density above the user's cap -> finding (editorial: the revision arm rewrites...)' (sweeps.md:636-639; mirrored at `nbt-skills/prompts/identify_issues.prompt.md:742-744`). The code-side tier system says otherwise: `FMT-P1` (em-dash density) and `FMT-P2` (spaced hyphen as dash) are absent from `FINDING_TIER_RULES` (`nbt_docx_format.py:595-608`) and therefore 'advisory' (`tier_of :611-613`), while `FMT-T1` (mixed quotation marks) IS `finding-tier`. The distinction is not cosmetic: the `DISPOSITION_MANDATE` makes the strictness of a row's disposal depend on exactly that tier column - 'a row whose tier column says finding is a defect an editor or a copyeditor would raise... no journal rule and editorial preference only are NOT dispositions for such a row' (`nbt_pipeline.py:2034-2038`) - and the seeded table prints the tier column right next to header text that calls the same rows 'findings for the revision/integration arms' (:6373-6374, :6390).

An agent that reads the header calls the row a finding and files it; an agent that reads the column calls it advisory and closes it with a one-liner; the repeated-disposition detector will then flag the `SECOND` agent's behaviour differently than the first's. This is the smallest of the audit's findings, but it is the cleanest instance of the operator's thesis - the same rule is described differently to different consumers, and the disagreement is measurable in disposition quality. Either `sweeps.md` should say 'advisory row: recorded, disposed, never gated' for `FMT-P1/P2`, or the code should classify them `finding-tier` - the current state has the skill and the scanner disagreeing about the same row in the same artifact.

5.3 C3 — Test-suite execution results

All 31 offline suites in `.nbt_test/` were executed in this environment (Python 3.12.14, Linux, umask 0002, no WSL interop). Twenty-nine pass in full. The two failures are environment-dependent, and both are test-side robustness gaps rather than pipeline defects - but they matter because the repository treats these suites as its regression safety net, and a net that fails on a plain Linux box cannot guard refactors made there.

Suite	Result in this environment	Root cause
29 of 31 suites (incl. <code>test_pipeline</code> , <code>test_grading_scheme</code> , <code>test_agent_consistency_2026_0922</code> , <code>test_anonymized_judging</code> , <code>test_evidence_pack</code>)	PASS	-

Suite	Result in this environment	Root cause
test_judge_blinding.py	1 check failed: 'uniform file modes'	Asserts hardcoded 0o644; umask 0002 yields 0o664. Should assert uniformity.
test_bug_audit_2026_0919.py	2 checks failed: both C16	docx2pdf.sh requires wslpath/cygpath (WSL-only); test lacks a skip guard.

Table 3: Offline suite results in the audit environment (Python 3.12.14, Linux, umask 0002, no WSL interop).

test_judge_blinding.py fails one check, 'uniform file modes', because it asserts the exact mode 0o644 ('modes == {0o644}', .nbt_test/test_judge_blinding.py:298) while this environment's umask 0002 produces 0o664. The blinding property the README actually promises - 'one mtime and one mode for the whole session, identical across all its views' - is about UNIFORMITY, and the observed set {0o664} is uniform; the assertion should check len(modes) == 1 (plus, optionally, that no mode carries more permission than the creating umask implies). test_bug_audit_2026_0919.py fails its two C16 checks with 'Error: neither wslpath nor cygpath available' - docx2pdf.sh is a WSL/Git-Bash tool by design (README: 'Word->PDF conversion via PowerShell (WSL/Git Bash)'), but the test does not skip when neither helper exists, so a legitimate platform limitation is reported as a failed regression. A skip-guard (the pattern the suite itself already uses for the live MCP handshake check in test_docx_converter.py) resolves it.

6. Remediation Roadmap

The fixes below are ordered by expected impact on the operator's stated symptoms. Each is scoped to be implementable within the existing architecture; none requires abandoning the round model, the blinding, or the judge contract. Effort is a coarse S/M/L estimate against the codebase's own conventions; risk names the main thing a fix could break.

#	Finding	Recommended fix	Effort	Risk
1	C1	In scan_format_in_sources, build the effective policy BEFORE the rebinding: policy = dict(getattr(mod, 'POLICY_DEFAULTS', {}) or {}); policy.update(policy or {}) with the PARAMETER captured under a different name (e.g. overrides = policy or {}). Add a regression test: seed a scan with max_em_dashes_per_1000=500 and assert no FMT-P1 row.	S	Low; seeded rows may change for operators who relied on the bug
2	A5	Add a JUDGE FEEDBACK step: after aggregation, collect every 'introduced' ledger item recorded against the round's CHAMPION across all judge sessions, deduplicate by (tier, evidence), and seed it into the next round's review sandbox as prior_round/judge_findings.json with the same reconciliation contract as PRIOR_ROUND_RULE. Nothing provenance-bearing needs to enter the judge sandbox - the flow is orchestrator-side.	M	Judge-to-review vocabulary mapping needs care (blinding is untouched)

#	Finding	Recommended fix	Effort	Risk
3	B1	Cache directed scores by (target_fingerprint, opponent_fingerprint) pair across rounds: when both members' content digests are unchanged, reuse the previous round's scores instead of re-running J sessions. Optionally carry the incumbent's own row forward when its content is unchanged.	M	Panel completeness logic must count reused scores as present
4	B2	Make the incumbent edge explicit: require vs_base >= 0 as an ELIGIBILITY gate for fresh arms (in addition to vs_original), OR rank on vs_base sign counts before the field-wide mean, with the exact sign test as the significance guard the code already computes.	S	2J-score noise: mitigate with the existing sign-test p
5	A4	Either (a) drop self-reported counts from the ranking key and let exact ties fall to the digest, or (b) make all arms equally verifiable: have the postcheck RE-DERIVE critical/writing counts from the frozen review plus the delivered package's own scans for every arm, not just the revise arm's reported-zero direction.	M	(b) requires a shared re-derivation; (a) accepts more incumbent retention
6	B3	Bound the field: keep only the previous round's champion (and the original) as persistent members; retire older pins to a reports-only role. Cuts judge cost from $O(R^3)$ to linear-per-round and removes the stale-opponent dilution of the median.	S	Loses some 'long memory' of past states; acceptable for a fixed-length run
7	B4	Route the rewrite feedback: archive rewritten/REWRITE_REPORT.md into reports/round_rewrite_feedback/ at round end (the archive helper already exists for review outputs) and seed its PROBLEMS SURFACED list into the next review's prior_round/ bundle, or into the SAME round's revise sandbox when timing allows.	S	Timing: same-round seeding needs the review to wait for rewrites or none
8	A1/A2	Re-price the mandated classes or de-mandate the work: either raise TIER_CAPS for formatting/writing to 2 with major rows allowed, or remove the L1-L11/M20-editorial mandates from the prompts and let the review findings drive all edits. Fix the WRITING_RUBRIC sentence 'ten unfixed rows are ten rows' to state the cap.	S	Calibration: the caps were an operator policy; changing them is a policy call
9	A6	Rebalance the auditor: allow 'drop - undecidable, escalate to human' as a third verdict for findings the auditor cannot decide (removing the confirm-ratchet), and forbid severity raises without a check id the judge contract recognises.	S	More human workload; reversible per finding
10	C2	Align the vocabulary: add FMT-P1/P2 to FINDING_TIER_RULES or change sweeps.md to call them advisory; one of the two must give.	S	Low
11	C3	Test hermeticity: assert mode UNIFORMITY rather than 0o644; add a wslpath/cygpath skip guard to C16.	S	None
12	A3	Add an inter-judge agreement diagnostic: for each pair judged in both directions, report the tier-agreement rate alongside direction_flips; surface systematic misclassification in DECISION_REPORT.md.	S	None (reporting only)

#	Finding	Recommended fix	Effort	Risk
1 3	B5	Replace K per-member integrations with ONE canonical integration into the round base plus (optionally) one variant arm; keep the difference ledger contract unchanged.	M	Loses the 'every member gets a merge' diversity argument

Table 4: Recommended fixes in suggested sequencing order. Effort: S = under a day, M = a few days within the existing architecture.

6.1 Sequencing and priorities

Sequencing. Items 1, 4, 6, 7, 10 and 11 are small, low-risk, and independently valuable; together they remove the dead knob, the wrong-edge gate, the cost explosion, the dead feedback channel and both test-suite false alarms - most of the mechanics behind 'slow convergence' and 'too much scoring'. Item 2 (judge feedback) and item 3 (verdict caching) are the structural investments: 2 converts the pipeline's most expensive artefact from a scalar into a steering signal, and 3 pays for it by eliminating the redundant re-judging that item 2's extra seeding would otherwise compound. Items 5 and 8 are policy decisions more than engineering ones and should be made with the operator's own calibration data (the 2026-09-17 panel) in view; the audit recommends (b) for 5 and the cap-raise variant for 8, but flags both as operator calls.

6.2 What not to change

What NOT to change. The blinding architecture is correct and well-executed; the derived-score straightjacket is the right idea with wrong constants; the content-addressed pin/verify discipline is a genuine strength (every integrity check the audit probed held). The pipeline's problem is not that its parts are unsound - it is that the parts were each locally optimised until their interfaces stopped agreeing: a scoring contract that prices out the prompts' mandates, an audit layer that feeds a tie-break the panel never sees, and a selection statistic that is not the improvement statistic. The fixes above all target interfaces, not parts.

7. Verification Log

Environment. The audit ran on Linux (Python 3.12.14, umask 0002, uid 1001, no WSL interop, no codex CLI) against the working tree of commit 7d19d80 (branch main) extracted from the uploaded archive. All file:line citations refer to that tree. Line numbers in `nbt_pipeline.py` were verified against the working tree, not the packed archive.

Test suite. All 31 suites were executed with the repository's own runner convention ('for t in `.nbt_test/test_*.py`; do python3 "\$t"; done') with a per-suite timeout of 300 seconds; total wall time approximately 12 minutes. Results are summarised in finding C3; the two failures print their own diagnosis lines, which are quoted there verbatim.

Runtime probes. The policy-shadowing probe (`scripts/probe_policy_shadow.py`) constructs a minimal `.docx` whose text carries 6 em-dashes in 23 words, imports `nbt_docx_format` directly, and shows that

analyse_package honours a custom max_em_dashes_per_1000 while the wrapper's parameter is dead. The chart figures (session counts, field sizes, corpus reads) were computed from the DEFAULTS block (nbt_pipeline.py:757-772) and the build_field/round_produced_ids definitions; the arithmetic is reproduced in the chart script (scripts/make_charts.py).

Searches. The claim that no stage consumes judge_review/ or REWRITE_REPORT.md downstream was established by full-text search of nbt_pipeline.py and both skills for every occurrence of the relevant filenames and artefact names, followed by reading each occurrence in context; the consumption sides (postcheck_judge, postcheck_rewrite, BOOKKEEPING_FILES, build_corpus_dir, _is_view_excluded_file) are cited in findings A5 and B4. The claim that the triage ledgers do not record findings A1-A3, A5-A6, B1-B5, C1-C2 rests on keyword searches of all three ledger files for each finding's subject matter; the two genuine overlaps (A4, B1's documentation) are marked in chapter 1.